K-ORCH: orquestrador para instâncias multikernel distribuídas

Eduardo Kaluf[1], **Carlos Alberto Maziero**[2]

[1] Departamento de Informática – Universidade Federal do Paraná (UFPR)
Curitiba – PR – Brazil

eduardo@kaluf.com.br, maziero@inf.ufpr.br

Abstract. *The multikernel architecture runs multiple independent kernel instances, each on a dedicated partition of cores and memory of the same processor, being a middle ground between containers and VMs and offering predictable performance, kernels with specialized configurations, and performance isolation between instances, without CPU and memory virtualization overhead. Current implementations of this architecture [Wang 2025b], manage these instances only within a single machine, and no tools exist to orchestrate them in distributed environments. This work proposes K-ORCH, an orchestrator that provisions, places, and monitors multikernel instances across multiple nodes, based on a desired-state model, agents responsible for partitioning cores and memory on demand, and placement policies driven by the available physical resources. K-ORCH will be experimentally evaluated through a quantitative analysis of provisioning time, control-plane overhead, density, and performance interference between instances, compared to orchestrated microVMs, in addition to comparing the orchestrator's own static and dynamic partitioning models. The expected outcome is to demonstrate the feasibility of distributed multikernel orchestration and to highlight its costs, advantages, and limits relative to current alternatives.*

Resumo. *A arquitetura multikernel executa múltiplas instâncias de kernel independentes, cada uma em uma partição dedicada de núcleos e memória de um mesmo processador, sendo um ponto intermediário entre containers e VMs, oferecendo desempenho previsível, kernels com configurações especializadas e isolamento de desempenho entre instâncias, sem o overhead de virtualização da CPU e da memória. As implementações desta arquitetura [Wang 2025b], gerenciam essas instâncias apenas dentro de uma única máquina, não existindo ferramentas que permitam a orquestração em ambientes distribuídos. Este trabalho propõe o K-ORCH, um orquestrador que provisiona, posiciona e monitora instâncias multikernel em múltiplos nós, a partir de um modelo de estado desejado, de agentes responsáveis por particionar núcleos e memória sob demanda e políticas de posicionamento com ênfase nos recursos físicos disponíveis. O K-ORCH será avaliado experimentalmente, em uma análise quantitativa, quanto ao tempo de provisionamento, ao custo do plano de controle, à densidade e à interferência de desempenho entre instâncias, em comparação com microVMs orquestradas, além de comparar modelos de particionamento estático e dinâmico do próprio orquestrador. Espera-se demonstrar a viabilidade da orquestração distribuída de multikernels, evidenciar custos, vantagens e limites diante das alternativas atuais.*

1. Introdução

O número de núcleos por processador cresce incessantemente, e processadores já chegam a ultrapassar uma centena de núcleos em um único chip [AMD 2024]. Os sistemas operacionais monolíticos, projetados em torno de estruturas de dados e controle compartilhadas por todos os núcleos, acompanharam esse crescimento à custa de mecanismos cada vez mais complexos, como travas de granularidade fina, estruturas de dados específicas por núcleo e políticas de afinidade e isolamento de CPUs [Boyd-Wickizer et al. 2010]. Essas soluções aumentam a complexidade, a superfície do kernel, e tornam o compartilhamento de estado entre núcleos um *bottleneck* para o desempenho e escalabilidade [Boyd-Wickizer et al. 2008]. Nesse contexto, surge a arquitetura multikernel, propondo tratar a máquina como uma rede de núcleos independentes, com estado replicado e comunicação explícita por mensagens, em vez de um único kernel com estado compartilhado, aproximando o próprio kernel a idea de um sistema distribuído [Baumann et al. 2009, Wentzlaff and Agarwal 2009].

Desde então, duas maneiras de subdividir logicamente os núcleos foram muito exploradas. A primeira reestrutura o SO por completo, como no Barrelfish, que executa um kernel por núcleo [Baumann et al. 2009]. A segunda, linha adotada por este trabalho, particiona a máquina física entre múltiplas instâncias de kernel independentes, com seus próprios núcleos e região de memória, tendo a possibilidade de possuir diferentes configurações e ajustes finos. Essa linha surgiu tendo em vista computação de alta performance (HPC), com sistemas que executam kernels leves ao lado do Linux [Shimosawa et al. 2014, Park et al. 2012, Gerofi et al. 2016a], e foi recentemente proposta para o próprio Linux, que passa a carregar instâncias independentes sobre núcleos e memória reservados, com realocação em tempo de execução [Wang 2025b, Wang 2025a]. Nesse modelo, um kernel hospedeiro particiona os recursos e inicia as instâncias, que se comunicam por troca de mensagens explícitas e por regiões de memória compartilhada, além disso, acessam o restante do hardware, em geral, por meio de dispositivos virtuais fornecidos pelo kernel hospedeiro. Neste trabalho, *instância multikernel* está relacionada a este conjunto de objetos: núcleos dedicados, uma região de memória própria, uma imagem da aplicação e uma configuração própria de kernel.

Ademais, a arquitetura multikernel ocupa um ponto único entre as duas formas de subdividir recursos que hoje são amplamente utilizadas. Containers são unidades executáveis de código que empacotam uma aplicação e suas dependências, utilizando mecanismos do próprio SO, como *namespaces* e *cgroups*, para dividir recursos e limitá-los. Containers são leves, porém compartilham o kernel do hospedeiro, isso amplia a superfície de ataque entre as aplicações e as deixam sujeitas à interferência de vizinhos (*noisy neighbors*). Por outro lado, VMs executam um kernel próprio sobre um hipervisor e oferecem isolamento mais forte, ao custo de um *overhead* pela comunicação hóspede-hospedeiro, E/S e acesso à memória, ou seja, possui consumo maior de recursos por instância. Dessa forma, uma instância multikernel entra como terceira via. Ela também executa um kernel próprio, mas diretamente sobre os núcleos físicos, sem possuir a camada de virtualização de CPU e memória das VMs, o que favorece um desempenho previsível e permite a especialização do kernel para cargas específicas de trabalho. Por outro lado, a alocação tem granularidade de núcleos por inteiro e o isolamento de segurança entre as instâncias fica dependente dos mecanismos de hardware da máquina.

A adoção de containers e VMs em larga escala está fortemente ligada com a criação de orquestradores que gerenciam essas unidades, dando destaque para a linha que foi do Borg ao Kubernetes [Burns et al. 2016]. As ferramentas existentes para multikernel, por outro lado, como a IHK [Shimosawa et al. 2014] e o kerf [Multikernel Technologies 2025], oferecem mecanismos para criar, configurar, monitorar e destruir instâncias, similares a ferramentas para containers como o *docker compose*, mas operam sobre uma única máquina, sem definir políticas sobre onde e quando essas instâncias devem ser criadas. Sem uma camada de orquestração robusta, a arquitetura tende a permanecer restrita a experimentos locais e teóricos, distante de ambientes distribuídos de produção como a nuvem, *fog computing* e a computação de borda.

Sendo assim, este trabalho propõe a ferramenta K-ORCH, um orquestrador para instâncias multikernel em nós distribuídos, capaz de suprir as especificidades desta arquitetura emergente. K-ORCH recebe pedidos de instâncias descritos com o estado desejado, escolhe um nó de destino com base nos núcleos e memória disponível, delega a um agente em cada nó o particionamento dos recursos e a inicialização da instância, além de monitorar as instâncias a fim de detectar e recuperar falhas. A proposta será avaliada quanto ao tempo de provisionamento, ao custo do plano de controle, à densidade e à interferência de desempenho entre instâncias, em comparação com microVMs orquestradas.

O restante deste documento está organizado da seguinte forma. A Seção 2 apresenta a justificativa do trabalho, a pergunta de pesquisa (Seção 2.1), os objetivos gerais (Seção 3) e específicos (Seção 3.1). A Seção 4 discute os trabalhos relacionados, e a Seção 5 descreve a metodologia. Por fim, a Seção 6 apresenta os resultados esperados, e a Seção 7, o cronograma de dois anos para a realização do trabalho.

2. Justificativa e Motivação

A disputa entre isolamento e desempenho descrito na Seção 1 tem consequências concretas em ambientes multi-tenant. Vulnerabilidades de escape de containers, como a CVE-2019-5736 e a CVE-2024-21626 [NIST 2019, NIST 2024], deixam evidente o risco de compartilhar o mesmo kernel entre cargas de diferentes clientes. Além disso, interferência entre cargas leva provedores a manter seus servidores subutilizados, com o intuito de preservar a latência de serviços críticos [Lo et al. 2015]. A indústria e a academia buscam soluções intermediárias para lidar com esses problemas, como o Firecracker [Agache et al. 2020], desenvolvido para executar funções e containers de diferentes clientes em microVMs, e o Kata Containers [OpenInfra Foundation 2026], que executa cada *pod* (unidade básico do *Kubernetes*) em uma máquina virtual leve. Esses esforços indicam que há demanda por alternativas aos dois modelos tradicionais, onde a arquitetura multikernel possui muito espaço para se estabilizar como uma terceira opção.

O multikernel é uma vantagem enorme para aplicações de larga escala em que configurações específicas do SO influenciam diretamente no desempenho das instâncias sendo executada, como em bancos de dados e monitores. Isso permite que não seja necessário dedicar uma máquina inteira especificamente para as aplicações, algo que não é possível com containers devido ao compartilhamento do kernel hospedeiro, evitando custos e dando maior velocidade a computação existente nestes sistemas. Em ambiente HPC, o uso de kernels leves ao lado do Linux já se mostrou uma alternativa viável em sistemas de produção massivos [Gerofi et al. 2016b]. Em computação de borda, onde o hardware

é limitado e a latência precisa ser previsível, a execução sem a camada de virtualização de CPU e memória também é uma grande vantagem. Ambos são cenários que a utilização de instâncias multikernel, com configurações específicas, beneficiariam os provedores.

A proposta de suporte a multikernel no Linux [Wang 2025b] aproxima a arquitetura do kernel mais utilizado em servidores, mas sem ferramentas que permitam expandi-la para além de uma máquina. Assim como a adoção de containers foi extremamente influenciada por orquestradores, a adoção do multikernel em ambientes distribuídos depende de uma camada equivalente. Mais que isso, do ponto de vista científico, caracterizar os custos e limites dessa orquestração é uma contribuição por si só, independentemente de como a arquitetura se comportar nos critérios propostos.

Uma alternativa à construção de um orquestrador próprio seria integrar instâncias multikernel ao *Kubernetes* por meio de uma API compatível com suas interfaces, de forma parecido ao que o Kata Containers realiza com suas VMs leves. Este trabalho opta por explorar um orquestrador dedicado principalmente porque o posicionamento depende de recursos físicos, como núcleos livres e memória reservada. Por isso, pretende-se investigar o particionamento sem estar amarrado a uma plataforma existente, além de que, um protótipo independente permite medir o custo da orquestração de forma isolada, com menos interferência e maior corretude.

2.1. Pergunta de Pesquisa

A pergunta central que funciona como base desse trabalho é: quais mecanismos são necessários para provisionar, posicionar e monitorar instâncias multikernel em múltiplos nós, e qual o custo desse gerenciamento em relação à orquestração de microVMs?

3. Objetivos

O objetivo geral deste trabalho é projetar, implementar e avaliar o K-ORCH, o qual deve ser capaz de provisionar, posicionar e monitorar instâncias multikernel em múltiplos nós.

3.1. Objetivo especificos

- Levantar o estado da arte: identificar os mecanismos dos orquestradores atuais e das ferramentas de gerência de multikernels, conectando as duas áreas, de modo a determinar quais mecanismos podem ser reaproveitados e quais precisam ser criados ou adaptados especificamente para o K-ORCH.
- Definir a unidade orquestrável: caracterizar a implementação multikernel que será utilizada, incluindo como núcleos e memória são reservados e realocados, e formalizar o que compõe uma instância e o modelo de estado desejado que o orquestrador deverá manter.
- Projetar a arquitetura: definir o funcionamento do plano de controle e sua interação com o plano de dados, projetar o agente responsável por instanciar as unidades multikernel a partir do kernel hospedeiro de cada nó e definir o modelo de comunicação entre os agentes e orquestrador.
- Implementar o ciclo de vida e o posicionamento das instâncias: implementar a criação, inicialização, parada e remoção das instâncias e o mecanismo para posicionamento, responsável por escolher o nó que atenderá cada requisição com base nos núcleos livres e na memória disponível.

- Desenvolver o modelo de particionamento de recursos: implementar um modelo híbrido, o qual permita a escolha entre reservar regiões de memória e CPU na inicialização de cada nó ou particionar dinamicamente a cada requisição.
- Implementar o monitoramento das instâncias: coletar métricas das máquinas do estado e do uso de recursos de cada nó, detectar falhas e perda de comunicação com o agente, utilizando o monitoramento para realizar a recuperação de instâncias com erros, reposicionando ou reiniciando conforme necessário.
- Avaliar experimentalmente o orquestrador: medir o tempo de provisionamento, o *overhead* do agente e do plano de controle, a densidade de instâncias por nó e a interferência de desempenho entre instâncias, comparando os resultados com microVMs orquestradas e comparar modelos de particionamento estático e dinâmico.

4. Trabalhos Relacionados

Esta seção apresenta os trabalhos correlatos, mostrando a evolução da arquitetura multikernel e conectando outros trabalhos ao foco desta proposta: provisionar, posicionar e monitorar instâncias multikernel entre diferentes nós. A Seção 4.1 apresenta a evolução das arquiteturas com múltiplos kernels, desde os sistemas para multiprocessadores de memória compartilhada até os multikernels utilizados em ambientes HPC e a proposta recente para o Linux. A Seção 4.2 discute como esses sistemas decidem a alocação de recursos, como os monitoram e como garantem o isolamento e a disponibilidade das instâncias. A Seção 4.3 aborda os orquestradores atuais e as ferramentas de gerência de multikernels.

4.1. Arquiteturas com múltiplos kernels

A ideia de estruturar o SO como um conjunto de kernels independentes surgiu há algumas décadas. O Hive [Chapin et al. 1995] organiza o SO como um sistema distribuído interno de kernels, denominados células, onde cada uma delas é responsável por um conjunto fixo de nós e opera como um SO independente, enquanto cooperam para oferecer às aplicações uma imagem única e transparente de sistema. A comunicação entre células ocorre por meio de *remote procedure calls* (RPC) e por leitura direta da memória de outras células.

Em vez de reescrever o SO por completo, o Disco [Bugnion et al. 1997] insere um pequeno monitor de máquinas virtuais que executa várias cópias de um SO na mesma máquina, comunicando-se por protocolos distribuídos comuns, como NFS e TCP/IP, o que permite executar sistemas operacionais especializados lado a lado com sistemas comerciais. O Cellular Disco [Govil et al. 2000] estende a proposta dividindo o próprio monitor em partes menores, a fim de conter falhas de hardware, transformando o multiprocessador no que seria um cluster virtual.

Já o Corey [Boyd-Wickizer et al. 2008] parte do princípio de que as aplicações devem controlar o compartilhamento. Por padrão, cada estrutura de dados do kernel é usada por um único núcleo, e abstrações como *address ranges*, *kernel cores* e *shares* permitem à aplicação indicar explicitamente o que deve ser compartilhado. Embora execute um único kernel, o Corey foi uma base importante para trabalhos posteriores, como o Barrelfish. O FOS [Wentzlaff and Agarwal 2009], por sua vez, propõe fatorar os serviços do SO em frotas de servidores distribuídos espacialmente pelos núcleos, com comunicação

exclusivamente por mensagens, motivado pela constatação de que o alocador de páginas físicas do Linux não escalava para além de oito núcleos. A comunicação por mensagens se consolidou nas propostas seguintes, incluindo a implementação de multikernel a partir do Linux [Wang 2025b].

A formulação moderna do conceito é apresentada por Baumann et al. [Baumann et al. 2009], que propõem tratar a máquina como uma rede de núcleos independentes. O sistema resultante, Barrelfish, executa um kernel por núcleo e segue três princípios: a comunicação entre núcleos é sempre explícita, a estrutura é neutra em relação ao hardware e o estado é replicado em vez de compartilhado, sendo mantido consistente por processos monitores. Em um trabalho anterior do mesmo grupo, Schüpbach et al. [Schüpbach et al. 2008] argumentam que a diversidade de hardware, nas formas de não uniformidade de memória, heterogeneidade de núcleos e diversidade entre sistemas, seria um desafio tão grande quanto a escalabilidade.

No ambiente HPC, a motivação principal é combinar o baixo *overhead* de um kernel reduzido, comumente denominado kernel leve, com a compatibilidade do Linux, e é dessa linha que deriva o modelo de instâncias particionadas adotado neste trabalho. O FusedOS [Park et al. 2012] particiona os núcleos entre o Linux e uma biblioteca derivada do kernel leve *Compute Node Kernel* (CNK), executada como processo de usuário e responsável por gerenciar os núcleos dedicados às aplicações. A *Interface for Heterogeneous Kernels* (IHK) [Shimosawa et al. 2014] generaliza essa estrutura em um modelo mestre-escravo, no qual o Linux, como kernel mestre, gerencia instâncias de kernels leves que se comunicam por uma camada própria. Ao comparar seis desses projetos, Gerofi et al. [Gerofi et al. 2016a] distinguem os sistemas com particionamento estático de recursos daqueles que permitem reparticionamento dinâmico, dimensão que este trabalho também investiga, agora em um ambiente com múltiplos nós.

Mais recentemente, a proposta de suporte a múltiplas instâncias do Linux em uma mesma máquina [Wang 2025b] carrega as instâncias via *kexec*, atribuindo a cada uma núcleos e memória que podem ser realocados em tempo de execução por meio de uma *device tree* e do mecanismo de *hotplug* [Wang 2025a]. Essa é a implementação da arquitetura prevista para o K-ORCH.

4.2. Balanceamento, observabilidade, isolamento e conceitos fundamentais

Decidir onde executar cada carga é um problema estudado há muito tempo em sistemas distribuídos. Casavant e Kuhl [Casavant and Kuhl 1988] propõem uma taxonomia hierárquica de escalonamento que distingue abordagens estáticas de dinâmicas e classifica estas quanto à distribuição da responsabilidade pelas decisões e à cooperação entre os nós. Shivaratri et al. [Shivaratri et al. 1992] decompõem os algoritmos de distribuição de carga em quatro políticas: de transferência (quando mover uma tarefa), de seleção (qual tarefa mover), de localização (para onde mover) e de informação (quando coletar o estado dos nós). Mitzenmacher [Mitzenmacher 2001] mostra que escolher a opção menos carregada entre apenas dois destinos aleatórios reduz exponencialmente a carga máxima esperada em relação à escolha puramente aleatória, resultado que fundamenta políticas simples e escaláveis.

Nos sistemas com múltiplos kernels, desde o início foi necessário decidir como distribuir os recursos da máquina entre as instâncias. No Hive, os mecanismos de compar-

tilhamento são implementados pela cooperação entre as células, mas a política é atribuída a um processo de usuário, o único com visão global do sistema, responsável por decidir de quais células alocar memória e como escalonar processos [Chapin et al. 1995]. No Disco, o monitor aplica políticas globais sobre as máquinas virtuais, movendo memória entre elas e escalonando dinamicamente as vCPUs para equilibrar a carga [Bugnion et al. 1997]. O Cellular Disco combina um balanceador ocioso, que usa apenas informação local, com um balanceador periódico, que consulta uma árvore de carga global [Govil et al. 2000]. No Barrelfish, uma base de conhecimento reúne informações do hardware e medições em tempo de execução, respondendo a consultas de otimização, como a alocação de drivers a núcleos [Baumann et al. 2009, Schüpbach et al. 2008]. Nos multikernels com foco em HPC, a gerência tende a ser exposta como uma interface. Na IHK, o kernel mestre assume o controle do mapeamento da memória física e oferece operações para reservar núcleos e memória, inicializar, consultar, desligar e destruir instâncias, porém não define políticas sobre quando utilizá-las [Shimosawa et al. 2014].

Decisões de posicionamento dependem de informação sobre o estado dos nós. Mansouri-Samani e Sloman [Mansouri-Samani and Sloman 1993] definem um modelo funcional de monitoramento de sistemas distribuídos dividido em quatro atividades (geração, processamento, disseminação e apresentação de informação) e apontam a desatualização da informação como o principal problema do monitoramento, o que afeta diretamente um orquestrador que decide com base em estado remoto. Em microsserviços, a observabilidade se organiza hoje em torno de três fontes: *metrics*, *logs* e *traces* [Faseeha et al. 2025]. Para métricas em clusters, o Ganglia [Massie et al. 2004] adota uma organização hierárquica, em que os nós de cada cluster trocam seus estados via *multicast* e clusters distintos são federados por uma árvore de conexões, privilegiando baixa sobrecarga por nó ao custo de métricas agregadas e menor granularidade temporal. O LDMS [Agelastos et al. 2014] mostra que é possível monitorar continuamente um sistema inteiro, mesmo em HPC, com processos coletores nos nós e processos agregadores que reúnem os dados, com baixa sobrecarga sobre as aplicações. Os próprios sistemas com múltiplos kernels também possuem mecanismos de monitoramento: a base de conhecimento do Barrelfish incorpora medições como a latência de comunicação entre pares de núcleos [Baumann et al. 2009], o balanceador periódico do Cellular Disco depende de uma árvore de carga atualizada constantemente [Govil et al. 2000] e o PIKA mede o tempo de cada aplicação para orientar suas escolhas [Beckmann et al. 2014]. Porém, essa informação é coletada e consumida em uma única máquina, sem compartilhamento entre os nós de um cluster.

Uma grande motivação para dividir o sistema em múltiplos kernels é a disponibilidade das aplicações. No Hive, uma falha de hardware ou de software afeta apenas os processos que usam os recursos da célula prejudicada. O sistema protege cada página contra escritas de outras células, descarta preventivamente as páginas que uma célula falha poderia ter corrompido e detecta falhas por heurísticas, cuja confirmação exige consenso das células sobreviventes [Chapin et al. 1995]. O Disco trata cada máquina virtual como unidade de contenção de falhas de software [Bugnion et al. 1997], e o Cellular Disco estende essa contenção a falhas de hardware: em centenas de experimentos com injeção de falhas, a recuperação foi bem-sucedida em todos os casos, em menos de meio segundo [Govil et al. 2000]. Além de conter falhas, a divisão em múltiplos kernels também é usada para isolar o desempenho das aplicações. O Pisces [Ouyang et al. 2015] permite

a coexistência, em um mesmo nó, de múltiplos ambientes de SO independentes, denominados enclaves, cada um executando um co-kernel leve que gerencia uma partição de recursos de hardware, tratando o isolamento de desempenho como requisito principal.

4.3. Orquestração

Os orquestradores atuais sofreram enorme influência da experiência do Google com o Borg, o qual foi consolidado no Kubernetes em um modelo declarativo, isto é, o usuário descreve o estado desejado, e laços de controle reconciliam continuamente o estado que está sendo observado com ele [Burns et al. 2016]. Uma revisão de orquestradores como Kubernetes, Docker Swarm e Apache Mesos identifica a escassez de propostas de orquestração orientada pela utilização de recursos e compara a adequação de cada ferramenta [Swetha et al. 2025]. Na mesma linha, uma implantação de contêineres em dois estágios distribui contêineres entre máquinas virtuais com granularidade grossa por meio de um algoritmo guloso, enquanto um algoritmo genético reparte os recursos dentro de cada máquina virtual [Lu et al. 2025], estrutura similar a escolher um nó e em seguida realizar o particionamento local, algo que será adotado neste trabalho.

Para cargas que exigem isolamento mais forte que o de contêineres, a abordagem predominante é integrar máquinas virtuais leves aos orquestradores existentes. O Firecracker [Agache et al. 2020] é um monitor de máquinas virtuais mínimo, desenvolvido para executar funções e contêineres de diferentes clientes em microVMs, e o Kata Containers [OpenInfra Foundation 2026] executa cada *pod* do Kubernetes em uma máquina virtual leve, integrado por meio da interface de runtime de contêineres (CRI). As microVMs orquestradas são, por isso, a referência de comparação adotada neste trabalho.

No contexto multikernel, a gerência das instâncias permanece local. No projeto Hobbes, um gerenciador de nó é responsável pelo ciclo de vida de enclaves distintos de um mesmo nó e pelos mecanismos de memória compartilhada entre eles [Kocoloski et al. 2015]. Para a proposta multikernel do Linux, a ferramenta kerf [Multikernel Technologies 2025] gerencia a criação, a configuração, o monitoramento e a destruição das instâncias, sendo a ferramenta mais próxima de um orquestrador, mas limitada a uma única máquina.

5. Metodologia

A metodologia utilizada por este trabalho tem caráter teórico-prático, combinando análise de algoritmos e mecanismos de orquestração, com suas respectivas implementações e uma avaliação experimental. A abordagem é quantitativa, baseada em identificar os benefícios e comparar o desempenho entre aplicações utilizando medições do comportamento do orquestrador e de instâncias sob cargas controladas. Por último, o nível da pesquisa é exploratório, testando e verificando diferentes possibilidades que possam ser utilizadas para gerenciar esse novo tipo de tecnologia, dado que a orquestração distribuída da arquitetura multikernel ainda não foi documentada na literatura.

5.1. Caracterizar e definir a unidade orquestrável

Na primeira etapa será levantado o estado da arte e a definição da unidade orquestrável. A plataforma multikernel será estudada experimentalmente quanto ao tempo de inicialização, à forma de reserva dos núcleos e da memória, ao acesso das instâncias à

rede, e a possíveis sinais disponíveis para detectar falhas. Com base nessa caracterização, será definido de maneira formal o que compõe uma instância (núcleos dedicados, região de memória, imagem e configuração do kernel) e o modelo de estado desejado mantido pelo orquestrador.

5.2. Projeto e implementação do protótipo

A segunda etapa realiza os objetivos de projeto da arquitetura e de implementação do ciclo de vida e do posicionamento. O plano de controle será implementado a fim de receber pedidos de instâncias, descritos por estado. O agente, executado no kernel de cada nó, será comandado pelo plano de controle, realizando particionamento de recursos e a inicialização, parada e remoção das instâncias, reportando o estado de cada uma delas. Inicialmente serão implementadas duas políticas básicas de posicionamento: *first-fit* e a escolha de um nó menos carregado entre dois sorteados [Mitzenmacher 2001].

5.3. Particionamento de recursos, monitoramento e recuperação

Na terceira etapa, será feito o particionamento de recursos e monitoramento. O orquestrador passará a operar em dois modos: estático, com partições predefinidas em cada nó, e dinâmico, com núcleos e memória particionados a cada pedido. O agente será um coletor periódico, abrangendo o estado e o uso de recursos de cada instância e dos nós, o plano de controle deve agregar essas informações e tomar decisões a partir delas. Falhas serão detectadas por ausência de sinais do agente e por eventos, a recuperação consistirá em reiniciar a instância no mesmo nó ou reposicioná-la em algum outro que esteja disponível e saudável.

5.4. Análise e refinamentos

A quarta etapa consiste em fazer a avaliação experimental e testar a ferramenta com experimentos específicos:

- **Provisionamento e custo do orquestrador:** tempo entre o pedido e resposta da instância, decomposto em particionamento, inicialização do kernel e avaliabilidade, comparado ao tempo de inicialização de microVMs. Analisar também consumo de CPU e memória do agente e do plano de controle.
- **Densidade e interferência:** executar cargas sensíveis à latência, *CPU bound*, *memory bound* ao lado de outras aplicações, comparando multikernel e microVMs.
- **Particionamento estático e dinâmico:** sequências de pedidos com diversos tamanhos diferentes, medindo a utilização dos núcleos e da memória de cada nó, pedidos rejeitados e tempo de provisionamento.

Com esses dados coletados, uma análise será feita e modificações mais específicas no orquestrador poderão ser realizadas, a fim de melhorar seu desempenho.

6. Resultados Esperados

O principal resultado esperado é um protótipo funcional da ferramenta K-ORCH, capaz de provisionar, posicionar e monitorar instâncias multikernel em múltiplos nós. A proposta busca entender e explorar os melhores caminhos para as especificidades da arquitetura multikernel, criando essa ferramenta robusta para a orquestração de sistemas distribuídos.

Almeja-se também, criar um histórico com os parâmetros utilizados e das maiores dificuldades para a realização da orquestração, mapeando e documentando, algoritmos, principais ideias, desempenhoe e outras métricas. Dessa forma, também será possível oferecer uma caracterização dos custos e limites da orquestração distribuída de multikernel.

7. Cronograma

A tabela 1 apresenta o cronograma organizado em 8 trimestres. Os trimestres um e dois tem como foco o estudo da plataforma multikernel e a definição formal da unidade orquestrável utilizada, criando um ambiente experimental reprodutível. Os trimestres três e quatro serão utilizados para projetar o funcionamento geral do orquestrador, e então, implementar os agentes de orquestração a fim de realizar o ciclo de vida das aplicações e o posicionamento, tendo como marco uma unidade funcional de orquestração ao fim do ano, servindo como prova de conceito. Os trimestres cinco e seis serão dedicados para o particionamento de recursos, o monitoramento e a recuperação de instâncias, além da preparação do ambiente para comparação com microVMs. Por último, nos trimestres sete e oito, será realizado testes e análises quanto ao desempenho da orquestração e das aplicações escolhidas, ou seja, uma avaliação experimental com foco em refinar o orquestrador e consolidar os resultados.

Tabela 1. Cronograma das tarefas organizado por trimestre.

Atividade	T1	T2	T3	T4	T5	T6	T7	T8
Levantamento do estado da arte	X	X						
Caracterização experimental da plataforma multikernel	X	X						
Definição da unidade orquestrável e modelo de estado		X						
Projeto da arquitetura			X					
Implementação do ciclo de vida e do posicionamento			X	X				
Implementação do particionamento de recursos					X	X		
Implementação do monitoramento e da recuperação de instâncias					X	X		
Preparação do ambiente de avaliação e das microVMs de referência					X	X		
Avaliação experimental e comparativa							X	X
Refinamento do orquestrador							X	X
Redação de artigos e do texto final				X		X	X	X

8. Considerações Finais

Este trabalho propõe o K-ORCH, um orquestrador que estende a gerência de instâncias multikernel, hoje restrita a uma única máquina, para ambientes com múltiplos nós. A proposta combina o modelo dos orquestradores atuais com os mecanismos de particionamento específicos à arquitetura multikernel e será avaliada diante microVMs orquestradas quanto ao custo de provisionamento, à densidade e à interferência de desempenho entre instâncias. A contribuição esperada é proporcionar uma terceira via para ambientes que

necessitam do melhor dos containers e das máquinas virtuais, proporcionando abstração suficiente a fim de escalar essa arquitetura para grandes sistemas de produção com facilidade e confiabilidade. Além disso, espera-se caracterizar custos e limites da orquestração distribuída da arquitetura multikernel.

Referências

Agache, A., Brooker, M., Florescu, A., Iordache, A., Liguori, A., Neugebauer, R., Piwonka, P., and Popa, D.-M. (2020). Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 419–434, Santa Clara - CA, USA.

Agelastos, A., Allan, B., Brandt, J., Cassella, P., Enos, J., Fullop, J., Gentile, A., Monk, S., Naksinehaboon, N., Ogden, J., Rajan, M., Showerman, M., Stevenson, J., Taerat, N., and Tucker, T. (2014). The lightweight distributed metric service: A scalable infrastructure for continuous monitoring of large scale computing systems and applications. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, pages 154–165, New Orleans - LA, USA.

AMD (2024). AMD EPYC 9005 series processors. Página do produto. Acessado em 23/10/2026.

Baumann, A., Barham, P., Dagand, P.-E., Harris, T., Isaacs, R., Peter, S., Roscoe, T., Schüpbach, A., and Singhania, A. (2009). The multikernel: A new OS architecture for scalable multicore systems. In *22nd ACM Symposium on Operating Systems Principles (SOSP)*, pages 29–44, Big Sky - MT, USA.

Beckmann, N. Z., Gruenwald III, C., Johnson, C. R., Kasture, H., Sironi, F., Agarwal, A., Kaashoek, M. F., and Zeldovich, N. (2014). PIKA: A network service for multikernel operating systems. Technical Report MIT-CSAIL-TR-2014-002, MIT Computer Science and Artificial Intelligence Laboratory, Cambridge - MA, USA.

Boyd-Wickizer, S., Chen, H., Chen, R., Mao, Y., Kaashoek, F., Morris, R., Pesterev, A., Stein, L., Wu, M., Dai, Y., Zhang, Y., and Zhang, Z. (2008). Corey: An operating system for many cores. In *8th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 43–57, San Diego - CA, USA.

Boyd-Wickizer, S., Clements, A. T., Mao, Y., Pesterev, A., Kaashoek, M. F., Morris, R., and Zeldovich, N. (2010). An analysis of Linux scalability to many cores. In *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Vancouver - BC, Canada.

Bugnion, E., Devine, S., Govil, K., and Rosenblum, M. (1997). Disco: Running commodity operating systems on scalable multiprocessors. *ACM Transactions on Computer Systems*, 15(4):412–447.

Burns, B., Grant, B., Oppenheimer, D., Brewer, E., and Wilkes, J. (2016). Borg, Omega, and Kubernetes. *ACM Queue*, 14(1).

Casavant, T. L. and Kuhl, J. G. (1988). A taxonomy of scheduling in general-purpose distributed computing systems. *IEEE Transactions on Software Engineering*, 14(2):141–154.

Chapin, J., Rosenblum, M., Devine, S., Lahiri, T., Teodosiu, D., and Gupta, A. (1995). Hive: Fault containment for shared-memory multiprocessors. In *15th ACM Symposium on Operating Systems Principles (SOSP)*, pages 12–25, Copper Mountain - CO, USA.

Faseeha, U., Syed, H. J., Samad, F., Zehra, S., and Ahmed, H. (2025). Observability in microservices: An in-depth exploration of frameworks, challenges, and deployment paradigms. *IEEE Access*, 13:72011–72039.

Gerofi, B., Ishikawa, Y., Riesen, R., Wisniewski, R. W., Park, Y., and Rosenburg, B. (2016a). A multi-kernel survey for high-performance computing. In *6th International Workshop on Runtime and Operating Systems for Supercomputers (ROSS)*, Kyoto - Japan.

Gerofi, B., Takagi, M., Hori, A., Nakamura, G., Shirasawa, T., and Ishikawa, Y. (2016b). On the scalability, performance isolation and device driver transparency of the IHK/McKernel hybrid lightweight kernel. In *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, Chicago - IL, USA.

Govil, K., Teodosiu, D., Huang, Y., and Rosenblum, M. (2000). Cellular Disco: Resource management using virtual clusters on shared-memory multiprocessors. *ACM Transactions on Computer Systems*, 18(3):229–262.

Kocoloski, B., Lange, J., Abbasi, H., Bernholdt, D. E., Jones, T. R., Dayal, J., Evans, N., Lang, M., Lofstead, J., Pedretti, K., and Bridges, P. G. (2015). System-level support for composition of applications. In *5th International Workshop on Runtime and Operating Systems for Supercomputers (ROSS)*, Portland - OR, USA.

Lo, D., Cheng, L., Govindaraju, R., Ranganathan, P., and Kozyrakis, C. (2015). Heracles: Improving resource efficiency at scale. In *42nd International Symposium on Computer Architecture (ISCA)*, pages 450–462, Portland - OR, USA.

Lu, C., Zhou, J., and Zou, Q. (2025). An optimized approach for container deployment driven by a two-stage load balancing mechanism. *PLOS ONE*, 20(1). Artigo e0317039.

Mansouri-Samani, M. and Sloman, M. (1993). Monitoring distributed systems. *IEEE Network*, 7(6):20–30.

Massie, M. L., Chun, B. N., and Culler, D. E. (2004). The Ganglia distributed monitoring system: Design, implementation, and experience. *Parallel Computing*, 30(7):817–840.

Mitzenmacher, M. (2001). The power of two choices in randomized load balancing. *IEEE Transactions on Parallel and Distributed Systems*, 12(10):1094–1104.

Multikernel Technologies (2025). kerf: Multikernel management system. `https://github.com/multikernel/kerf`. Acessado em 10/10/2026.

NIST (2019). CVE-2019-5736: runc container breakout. `https://nvd.nist.gov/vuln/detail/CVE-2019-5736`. National Vulnerability Database. Acessado em 23/10/2026.

NIST (2024). CVE-2024-21626: runc container breakout. `https://nvd.nist.gov/vuln/detail/CVE-2024-21626`. National Vulnerability Database. Acessado em 23/10/2026.

OpenInfra Foundation (2026). Kata Containers: The speed of containers, the security of VMs. `https://katacontainers.io`. Acessado em 23/10/2026.

Ouyang, J., Kocoloski, B., Lange, J. R., and Pedretti, K. (2015). Achieving performance isolation with lightweight co-kernels. In *24th International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*, pages 149–160, Portland - OR, USA.

Park, Y., Van Hensbergen, E., Hillenbrand, M., Inglett, T., Rosenburg, B., Ryu, K. D., and Wisniewski, R. W. (2012). FusedOS: Fusing LWK performance with FWK functionality in a heterogeneous environment. In *24th International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*, pages 211–218, New York - NY, USA.

Schüpbach, A., Peter, S., Baumann, A., Roscoe, T., Barham, P., Harris, T., and Isaacs, R. (2008). Embracing diversity in the Barrelfish manycore operating system. In *Workshop on Managed Many-Core Systems (MMCS)*, Boston - MA, USA.

Shimosawa, T., Gerofi, B., Takagi, M., Nakamura, G., Shirasawa, T., Saeki, Y., Shimizu, M., Hori, A., and Ishikawa, Y. (2014). Interface for heterogeneous kernels: A framework to enable hybrid OS designs targeting high performance computing on manycore architectures. In *21st International Conference on High Performance Computing (HiPC)*, pages 1–10, Goa - India.

Shivaratri, N. G., Krueger, P., and Singhal, M. (1992). Load distributing for locally distributed systems. *Computer*, 25(12):33–44.

Swetha, R., Thriveni, J., and Venugopal, K. R. (2025). Resource utilization-based container orchestration: Closing the gap for enhanced cloud application performance. *SN Computer Science*, 6. Artigo 191.

Wang, C. (2025a). Multikernel: Kernel-to-kernel isolation with elastic resource management. `https://lpc.events/event/19/contributions/2074/`. Linux Plumbers Conference. Acessado em 10/10/2026.

Wang, C. (2025b). [RFC patch 0/7] kernel: Introduce multikernel architecture support. `https://lore.kernel.org/lkml/20250918222607.186488-1-xiyou.wangcong@gmail.com/`. Linux Kernel Mailing List. Acessado em 10/10/2026.

Wentzlaff, D. and Agarwal, A. (2009). Factored operating systems (fos): The case for a scalable operating system for multicores. *ACM SIGOPS Operating Systems Review*, 43(2):76–85.